FIAP ON

BUILDING STATISTICAL MODELS

ANÁLISE MULTIVARIADA DE DADOS

4

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – ACP na prática 8
Figura 2 – Matriz de correlação genérica 9
Figura 3 – Fórmula autovalores e autovetores 10
Figura 4 – Fórmula autovalores e autovetores com números 10
Figura 5 – Etapas do ACP 11
Figura 6 – Exemplo *Screeplot* 12
Figura 7 – Dados municípios 16
Figura 8 – Correlação das variáveis 17
Figura 9 – Dados selecionados 18
Figura 10 – Dados normalizados 19
Figura 11 – Resultados autovalores 20
Figura 12 – Análise do *screeplot* 21
Figura 13 – Variância explicada demonstrada 22
Figura 14 – Gráfico das cargas modelo retreinado 23
Figura 15 – Resultados cargas fatoriais *FactorAnalysis* 25
Figura 16 – Componentes adicionados aos dados 26

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Ferramentas Python ACP.....14
Quadro 2 – Descrição das variáveis.....16

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Importar módulos iniciais e dados.....15
Código-fonte 2 – Dados dos exemplos 15
Código-fonte 3 – Obter correlação das variáveis 17
Código-fonte 4 – Seleção das variáveis 18
Código-fonte 5 – Processo para normalizar os dados.....19
Código-fonte 6 – Aplicar o ACP.....19
Código-fonte 7 – Obter os autovalores.....20
Código-fonte 8 – Obtenção do *screeplot* 20
Código-fonte 9 – Obter variância explicada.....21
Código-fonte 10 – Obter a ACP retreinada 23
Código-fonte 11 – Obtenção das cargas fatoriais 23
Código-fonte 12 – Obtenção dos componentes com *FactorAnalysis* 24
Código-fonte 13 – Obtenção das cargas fatoriais com *FactorAnalysis* 24
Código-fonte 14 – Gerar gráfico das cargas fatoriais com *FactorAnalysis* 24
Código-fonte 15 – Adicionar os componentes no *dataframe* 25

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 ANÁLISE MULTIVARIADA DE DADOS 6
1.1 Introdução 6
1.2 Principais conceitos sobre análise de componentes principais 7
1.2.1 Autovalores e autovetores.....9
1.3 Procedimentos para realizar uma análise de componentes principais 11
2 APLICAÇÕES PRÁTICAS EM ANÁLISE DE COMPONENTES PRINCIPAIS 14
2.1 Principais funções no Python 14
2.2 Dados e exemplos.....15
2.3 Etapa 1: avaliar correlações 17
2.4 Etapa 2: definir o número de componentes.....18
2.4.1 Critério do autovalor 20
2.4.2 Critério *screeplot* 20
2.4.3 Critério da variância explicada 21
2.5 Etapa 3: interpretação 22
ATIVIDADE TREINO – ANÁLISE DE COMPONENTES PRINCIPAIS.....27
REFERÊNCIAS.....29

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 ANÁLISE MULTIVARIADA DE DADOS

1.1 Introdução

Muitas vezes o desafio da área de negócio não é explicar se uma variável objetiva foi importante ou avaliar se alguma determinada ação foi sucesso ou fracasso. Em certas situações, torna-se necessário encontrar padrões novos nos dados que ainda não foram percebidos ou criar índices para resumir a grande quantidade de colunas existentes em um banco de dados.

Diante disso, a área de *Analytics* precisará de técnicas específicas para auxiliar na resolução desses problemas de negócios. Nesses momentos, entram em cena os modelos de *Machine Learning* não supervisionados.

No caso dessas técnicas, somente os inputs de entrada são utilizados para o processo de aprendizagem. Deste modo, não há um *output* para ser avaliado, mas as saídas são utilizadas para descobrir algum padrão ou subgrupos nos dados.

Neste capítulo, conheceremos uma das técnicas mais importantes nesse desafio. A análise de componentes principais (ACP) poderá ser utilizada tanto para resumir quanto para reduzir a quantidade de informações de um banco de dados e transformar em índices mais simples de serem explicados.

Conforme Jollife e Cadima (2016), a ACP é uma das técnicas de aprendizagem não supervisionada mais antigas e uma das mais utilizadas. Essa técnica foi desenvolvida por Karl Pearson em 1901 com objetivos matemáticos, visando ajustar linhas e planos em um sistema de pontos.

Depois disso, Fisher realizou uma breve aplicação da técnica em análise de dados. Porém, foi Harold Hotelling, alguns anos depois, que desenvolveu toda a metodologia de como a ACP é realizada nos dias atuais. Um ponto interessante a ser ressaltado é que a técnica não pode ser realizada por vários anos desde a criação devido à complexidade dos cálculos a serem realizados.

Somente após o desenvolvimento dos computadores foi possível aplicar a técnica em bancos de dados, uma vez que era necessária uma capacidade maior de

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

memória para armazenar os dados de todas as colunas, realizar todas as estimações necessárias e depois reduzir a dimensão dos dados.

Nos próximos tópicos, explicaremos o conceito da análise de componentes principais, a forma de aplicação em análise de dados e como executar a técnica no ambiente do Python.

1.2 Principais conceitos sobre análise de componentes principais

Como produto deste capítulo, no mínimo duas perguntas deverão ser respondidas ao fim da leitura, são elas:

- **O que seria reduzir dimensão de dados?**
- **Quando seria o caso de realizar esse processo?**

Conforme Maroco (2010), a Análise de Componentes principais (ACP) é uma técnica voltada a transformar um conjunto de variáveis correlacionadas em um conjunto menor de dimensões. Ou seja, a nova dimensão (o componente principal) é uma combinação linear das variáveis originais.

Tomamos como exemplo o Índice de Desenvolvimento Humano (IDH), que mede o quanto uma região está evoluída quanto à educação, renda e saúde. Vamos assumir que o banco de dados tenha essas três colunas, mas queremos transformá-las em um único índice, como conseguiríamos transformar essas três colunas em uma única denominada IDH?

Em um caso simplista, poderíamos basicamente fazer uma média das três colunas e, no final, criaríamos o IDH. De certo modo, isso é uma forma de reduzir três dimensões (educação, renda e saúde) em uma única medida, o IDH. Porém, não estaríamos considerando a possível correlação existente entre as variáveis, fazendo com que possamos induzir diversos erros de medidas, tornando a mensuração do IDH pouco confiável.

Deste modo, a ACP permite que as dimensões sejam reduzidas considerando a média ponderada, sendo que os pesos são obtidos por meio da correlação entre as variáveis utilizadas. Por meio do uso dessa técnica, podem-se criar métricas de avaliação mais confiáveis e que poderão ser aplicadas em diversas análises de dados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Diante disso, do ponto de vista prático, reduzir dimensão nada mais é do que, em vez de utilizar, por exemplo, dez colunas de um banco de dados para explicar uma variável independente, utilizaríamos somente duas, mas essas novas dimensões explicam a variável independente tão bem quanto as dez anteriores.

Assim, em vez de explicar dez variáveis em uma reunião de negócios, poderemos ter os mesmos, ou até melhores, resultados quando apresentarmos somente dois índices criados das dez dimensões anteriores.

Na Figura “ACP na prática”, apresentamos como foi o processo de transformar oito indicadores de qualidade de vida em duas dimensões que posteriormente poderíamos chamar de “Qualidade de estilo de vida” (pca1) e “Qualidade de educação” (pca2).

Todas as 8 dimensões

Reduzidas nestes 2 componentes

t_agua	t_banagua	t_lixo	t_luz	T_ANALF18M	T_ATRASO_2_BASICO	T_ATRASO_2_FUND	T_ATRASO_2_MED	pca1	pca2
93.69	80.20	94.05	93.98	13.00	22.03	17.82	7.79	0.04331537	0.10065941
98.54	85.35	96.72	98.58	6.53	19.35	16.74	3.96	-0.28593668	0.44114744
95.49	91.63	99.14	96.38	14.73	15.59	12.93	6.93	-0.30413789	0.40889370
97.96	93.44	98.17	98.89	8.94	15.57	12.58	7.33	-0.31907779	0.65356849
97.53	86.87	91.99	98.85	11.17	12.93	10.27	3.82	-1.03020436	-0.05243995
95.54	93.89	92.79	97.76	13.41	13.74	9.80	5.02	-0.87138304	0.04706521

Figura 1 – ACP na prática
Fonte: Elaborado pelo autor (2022)

Outro ponto importante seria entender quando aplicar essa técnica, uma vez que não é em todo caso que seria a melhor opção. O primeiro passo a ser entendido é que a ACP é aplicada somente nas colunas do banco de dados, ou seja, caso o objetivo seja reduzir o número de linhas ou encontrar similaridade entre dois indivíduos, essa técnica não deve ser aplicada, porque não atingirá o objetivo buscado. Nesses casos, uma análise de conglomerados *(Cluster)* seria a melhor saída.

Assumindo que o objetivo seja reduzir a dimensão das colunas, precisaríamos verificar a correlação entre essas variáveis. Caso os coeficientes sejam muito baixos, também não devemos utilizar essa técnica.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Desta forma, a ACP será utilizada quando tivermos variáveis correlacionadas no banco de dados e reduzir essa dimensão trará ganhos tanto de entendimento e explicação de um problema quanto de performance computacional.

Conforme Hair *et al.* (2009), a ACP será de grande importância quando buscamos extrair o número mínimo de fatores, porém significativos, para explicar toda a variabilidade existente nos dados originais.

Um ponto que levanta a curiosidade de muitos analistas é o processo de reduzir a dimensão, isto é, como os pesos são obtidos, dado que ferramentas, como o Python, praticamente trazem os resultados prontos, fazendo com que muitos estudantes acreditem que algum processo “mágico” foi realizado. No próximo tópico, vamos tentar resumir o processo matemático que ocorre nas ferramentas.

1.2.1 Autovalores e autovetores

Reduzir a dimensão de um conjunto de dados não é um processo “mágico”, conceitos relativamente avançados relacionados à Álgebra Linear são aplicados e, deste modo, conseguimos obter a tão buscada redução de dimensão. Esse processo é obtido por meio da extração de autovalores *(Eigenvalues)* e autovetores *(Eigenvectors).*

Vejamos uma matriz de correlação genérica apresentada na Figura “Matriz de correlação genérica”. Nosso objetivo é reduzir a dimensão dessa matriz.

	Var1	Var2
Var1	1	0,9
Var2	0,9	1

Figura 2 – Matriz de correlação genérica
Fonte: Elaborado pelo autor (2022)

Primeiramente, percebemos que existe uma correlação muito alta entre “Var1” e “Var2”, ou seja, a técnica pode ser aplicada. Agora podemos utilizar um conceito de

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Álgebra Linear, a decomposição de matrizes. Esse conceito indica que uma matriz pode ser decomposta em outras, mas toda a informação armazenada será mantida.

Para realizar esse processo de decomposição, podemos resolver uma equação observada na Figura “Fórmula autovalores e autovetores”.

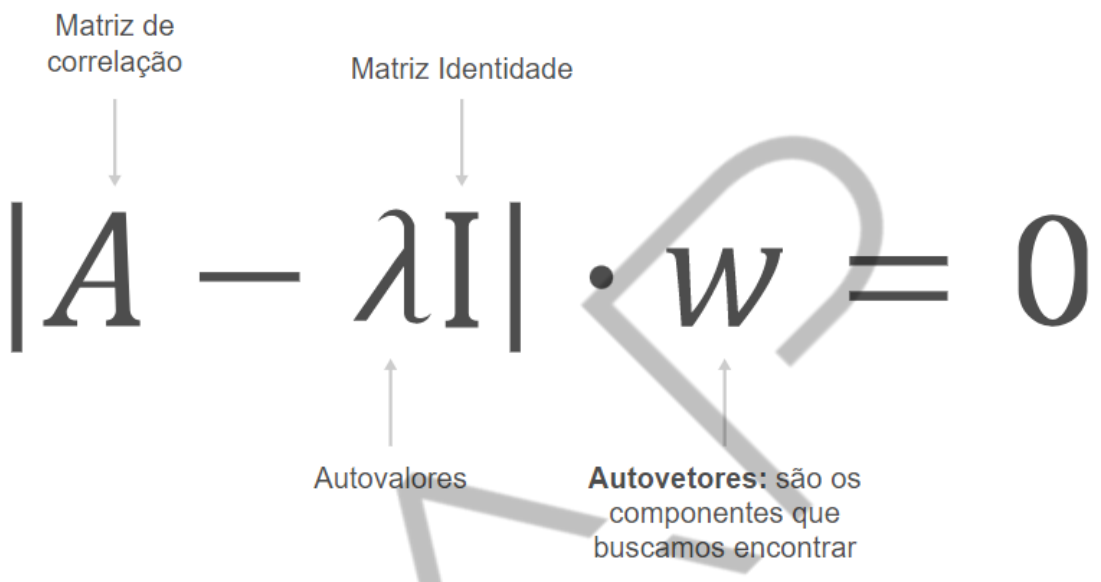

Figura 3 – Fórmula autovalores e autovetores
Fonte: Elaborado pelo autor (2022)

Conforme podemos observar na figura, esse problema matemático busca realizar a subtração entre uma matriz de correlação e uma matriz identidade e fazer com que esse valor seja zero. Para atingir esse objetivo, buscou-se propor o ajuste de dois elementos para ponderar a relação, os autovalores (λ) e os autovetores (w). Para simplificar, vejamos a Figura “Fórmula autovalores e autovetores com números”.

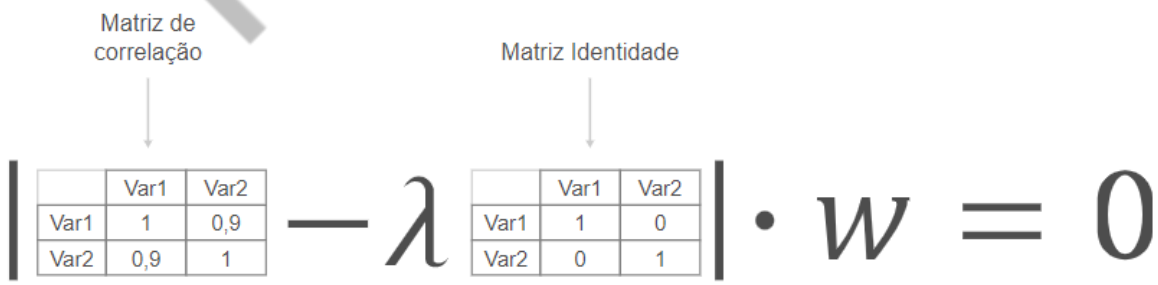

Figura 4 – Fórmula autovalores e autovetores com números
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Existem diversas ferramentas do Python que buscarão resolver essa fórmula. O analista adicionará como *input* a matriz de correlação, a ferramenta criará uma matriz de identidade, os autovalores (λ) e os autovetores (w) serão obtidos por otimização numérica. Nesse processo, teremos a oportunidade de reduzir a dimensão dos dados. Um detalhe interessante é que os valores de w são exatamente os valores das variáveis resumidos.

Para realizar uma análise de componentes principais, não basta simplesmente buscar os dados e rodar o programa no Python, por exemplo. Torna-se necessário seguir uma rigorosa sequência de etapas que auxiliará o analista a tomar as melhores decisões. No próximo tópico, detalharemos a sequência de etapas que poderão ser utilizadas.

1.3 Procedimentos para realizar uma análise de componentes principais

Após o entendimento dos conceitos iniciais sobre ACP, vamos entender o processo de realização das análises. As etapas a serem apresentadas foram baseadas nas proposições de Hair *et al.* (2009) e Maroco (2010); no caso, teremos duas etapas essenciais e uma opcional, são elas:

1. Gerar e analisar a matriz de correlação.
2. Definir o número de componentes a serem extraídos.
3. Interpretação dos componentes (opcional).

Na Figura “Etapas do ACP”, apresentamos uma visualização das etapas principais do ACP. Podemos perceber as análises mais importantes de forma visual, isso facilita relembrar esses passos em certas situações do dia a dia do analista.

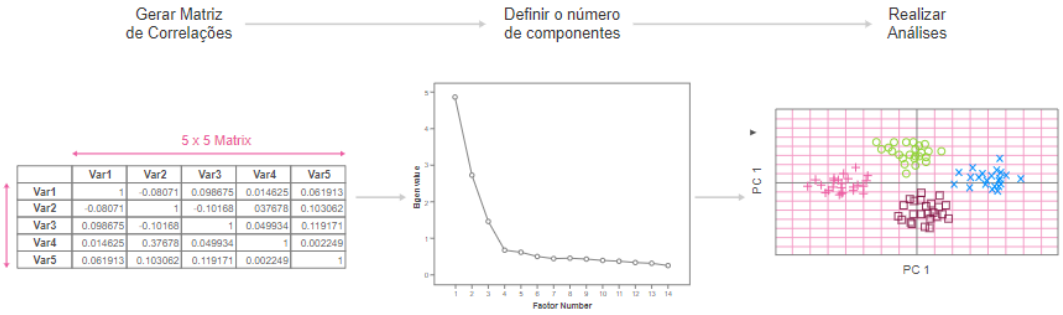

Figura 5 – Etapas do ACP
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A primeira etapa, que seria analisar a matriz de correlações, é essencial, somente após essa análise podemos definir a possibilidade de realizar a ACP ou não. A dica nesse estágio é verificar as correlações em módulo (sem considerar o sinal), buscando encontrar valores moderados a altos. Quanto mais correlações altas são encontradas, melhor será a qualidade da ACP.

Uma vez que podemos realizar uma ACP, avançamos para a segunda etapa; no caso, precisamos definir o quanto possível as dimensões serão reduzidas. Um detalhe importante é que o número máximo de componentes é igual ao número de variáveis. Porém, os primeiros componentes possuem informação suficiente para explicar a variabilidade dos dados.

Deste modo, podemos apresentar três critérios: autovalores, gráfico de “cotovelo” (*screeplot*) e percentual da variância explicada. No caso do critério de Autovalores, buscaremos observar quais componentes possuem autovalores acima de 1, caso tenham, esses componentes serão importantes para reduzir a dimensão.

No caso do *screeplot*, utilizamos uma técnica de observar em um gráfico os autovalores ordenados do maior para o menor, conforme podemos observar na Figura “Exemplo *screeplot*”.

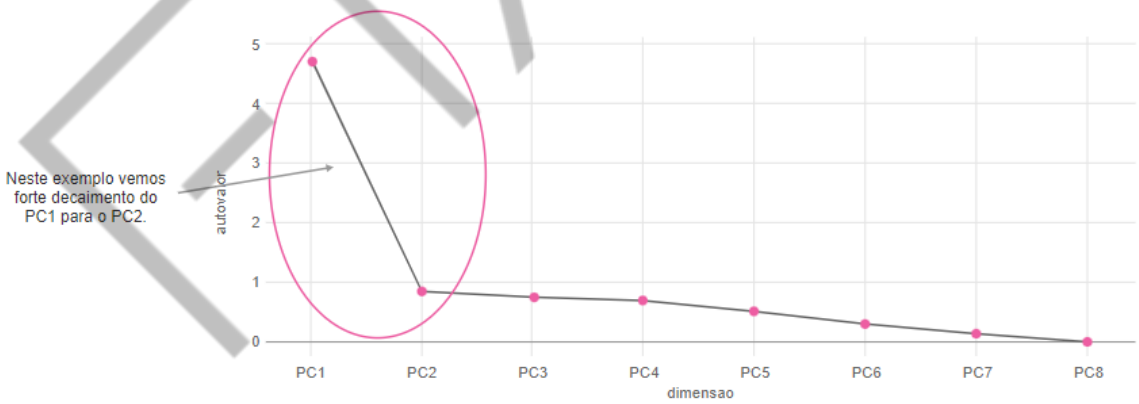

Figura 6 – Exemplo *screeplot*
Fonte: Elaborado pelo autor (2022)

Para analisar o gráfico, podemos seguir duas regras: a primeira, novamente verificar quais autovalores são maiores que 1. Na segunda, podemos verificar o decaimento do gráfico. Enquanto o gráfico mostrar forte decaimento nos autovalores, devemos manter esses componentes. No exemplo da figura, oito dimensões poderão ser reduzidas a um ou dois componentes.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O terceiro critério é o da variância explicada. O cálculo é bastante simples, teremos que gerar um quociente entre o autovalor e o número total de dimensões. Por exemplo, imagine que os autovalores dos componentes principais 1 e 2 (PC1 e PC2) foram os números **4,7** e **0,85**; temos um total de oito colunas.

Fazemos os seguintes quocientes: **4,7/8** = **0,59** e **0,85/8** = **0,11**. Caso mantivermos somente o PC1, teremos **59%** da variância explicada; caso precisarmos manter PC1 e PC2, traremos **69%** de informação nos dados.

Como valores-referência, em ciências sociais (psicologia, administração e economia), podemos manter entre mínimo 60% e razoável 70%. Nas engenharias e saúde, exige-se algo em torno de 80% a 90%. Todos esses valores podem ser obtidos tranquilamente no Python.

Conforme explicamos anteriormente, a terceira etapa é opcional, uma vez que dependerá do objetivo do analista. Por exemplo, caso o objetivo seja de utilizar um *Machine Learning* supervisionado, não é obrigatório interpretar, dado que os componentes serão utilizados para melhorar a capacidade de classificação ou preditiva dos algoritmos supervisionados.

Contudo, caso o objetivo seja visualização e criação de índices, torna-se importante interpretar o significado dos componentes. Nesse momento, devemos entender as cargas fatoriais de cada fator.

As cargas fatoriais nada mais são do que a correlação entre o componente e as variáveis utilizadas, quanto maior a carga, maior a correlação do componente com a variável.

Após essa longa explanação, o foco dos próximos tópicos será prático. Entenderemos como, por meio do Python, conseguimos aplicar a técnica do ACP.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 APLICAÇÕES PRÁTICAS EM ANÁLISE DE COMPONENTES PRINCIPAIS

Nos próximos tópicos, serão trabalhados em mais detalhes os conceitos citados anteriormente. Além disso, aplicaremos algumas das ferramentas como exemplo prático no Python.

Deste modo, vamos conhecer as principais funções, os dados e aplicar o algoritmo de componentes principais.

2.1 Principais funções no Python

Como ferramentas necessárias, os módulos Numpy e Pandas serão as principais bibliotecas do Python novamente. As bibliotecas Matplotlib e Seaborn serão novamente utilizadas para análise gráfica.

No caso da análise de componentes principais, trabalharemos com o módulo *scikit-learn*. Essa ferramenta será usada para normalizar os dados, aplicar a ACP e executar a função *FactorAnalysis* para auxiliar na interpretação dos componentes.

No Quadro "Ferramentas Python ACP", apresentamos um resumo das principais funções que serão utilizadas nos exemplos.

Módulos	Funções	Aplicações
Matplotlib	Várias funções serão utilizadas	Análise gráfica
Seaborn	Várias funções serão utilizadas	Análise gráfica
Numpy	Várias funções serão utilizadas	Análise numérica
Pandas	Várias funções serão utilizadas	Leitura e tratamento de dados
Scikit-Learn	scale	Normalizar os dados
	PCA	Aplicar a ACP
	FactorAnalysis	Auxiliar na interpretação dos componentes

Quadro 1 – Ferramentas Python ACP
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Dados e exemplos

Novamente utilizaremos o *Colab* digitando no navegador https://colab.research.google.com/, criaremos um notebook e iniciaremos as análises. Nos exemplos a seguir, utilizaremos os dados “dados_municipios.csv”. O objetivo é criar índices de avaliação de desenvolvimento para os municípios do Brasil utilizando dados do censo de 2010.

Os códigos para importar os módulos e carregar os dados podem ser vistos no Código-fonte “Importar módulos iniciais e dados”. Juntamente aos códigos apresentados neste texto, também é possível analisar os códigos de forma mais detalhada no arquivo “cap9_exemplos.ipynb” disponibilizado.

```
# gerais
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# pca
from sklearn.preprocessing import scale
from sklearn.decomposition import PCA
from sklearn.decomposition import FactorAnalysis

# carregar dados
dados = pd.read_csv('dados_municipios.csv')
```

Código-fonte 1 – Importar módulos iniciais e dados
Fonte: Elaborado pelo autor (2022)

No Código-fonte “Dados dos exemplos”, podemos observar como obter as cinco primeiras linhas do *dataset* que será utilizado.

```
# dados municipios
dados.head()
```

Código-fonte 2 – Dados dos exemplos
Fonte: Elaborado pelo autor (2022)

Na Figura “Dados municípios”, podemos observar as cinco primeiras linhas do *dataset*. Temos 11 variáveis, o objetivo é criar índices combinando os dados das seguintes variáveis: t_agua, t_banagua, t_lixo, t_luz, T_ANALF18M, T_ATRASO_2_BASICO, T_ATRASO_2_FUND, T_ATRASO_2_MED.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ano	estado	municipio	t_agua	t_banagua	t_lixo	t_luz	T_ANALF18M	T_ATRASO_2_BASICO	T_ATRASO_2_FUND	T_ATRASO_2_MED
2010	Rondônia	ALTA FLORESTA D'OESTE	93.69	80.20	94.05	93.98	13.00	22.03	17.82	7.79
2010	Rondônia	ARIQUEMES	98.54	85.35	96.72	98.58	8.53	19.35	16.74	3.96
2010	Rondônia	CABIXI	95.49	91.63	99.14	96.36	14.73	15.59	12.93	6.93
2010	Rondônia	CACOAL	97.96	93.44	98.17	98.89	8.94	15.57	12.58	7.33
2010	Rondônia	CEREJEIRAS	97.53	86.87	91.89	98.85	11.17	12.93	10.27	3.82

Figura 7 – Dados municípios
Fonte: Elaborado pelo autor (2022)

No Quadro “Descrição das variáveis”, apresentamos o significado de cada indicador de desenvolvimento obtido no censo de 2010.

Variáveis	Significado
ano	Ano em que o dado foi obtido
estado	UF em que ocorreu a pesquisa
municipio	Município em que ocorreu a pesquisa
t_agua	Percentual da população que vive em domicílios com água encanada
t_banagua	Percentual da população que vive em domicílios com banheiro e água encanada
t_lixo	Percentual da população que vive em domicílios urbanos com serviço de coleta de lixo
t_luz	Percentual da população que vive em domicílios com energia elétrica
T_ANALF18M	Taxa de analfabetismo da população de 18 anos ou mais de idade
T_ATRASO_2_BASICO	Percentual da população de 6 a 17 anos de idade frequentando o ensino básico que tem 2 anos ou mais de atraso idade-série
T_ATRASO_2_FUND	Percentual da população de 6 a 14 anos de idade frequentando o ensino fundamental que tem 2 anos ou mais de atraso idade-série
T_ATRASO_2_MED	Percentual da população de 15 a 17 anos de idade frequentando o ensino médio que tem 2 anos de atraso idade-série

Quadro 2 – Descrição das variáveis
Fonte: Elaborado pelo autor (2022)

Nos próximos tópicos, apresentaremos as análises seguindo as três etapas para aplicação do PCA no Python.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.3 Etapa 1: avaliar correlações

Conforme foi descrito nas etapas para realização de uma ACP, vamos iniciar pela avaliação das correlações das variáveis. No Código-fonte “Obter correlação das variáveis”, apresentamos o processo para observar as correlações entre os indicadores de desenvolvimento.

```
# obter correlacoes
colunas_correl = ['t_agua', 't_banagua', 't_lixo', 't_luz',
                  'T_ANALF18M', 'T_ATRASO_2_BASICO', 'T_ATRASO_2_FUND'
, 'T_ATRASO_2_MED']
correlacao = dados[colunas_correl].corr()

# grafico correlacao
correlacao.style.background_gradient(cmap='coolwarm').set_precision(2)
```

Código-fonte 3 – Obter correlação das variáveis
Fonte: Elaborado pelo autor (2022)

Podemos perceber que selecionamos as variáveis, depois obteve-se a correlação pelo comando “corr” e, por último, criamos um gráfico que facilita a leitura das correlações. Os resultados são observados na Figura “Correlação das variáveis”, quanto mais perto de 1, mais vermelho será o valor; quanto mais perto de -1, mais azulado.

	t_agua	t_banagua	t_lixo	t_luz	T_ANALF18M	T_ATRASO_2_BASICO	T_ATRASO_2_FUND	T_ATRASO_2_MED
t_agua	1.00	0.72	0.28	0.41	-0.63	-0.54	-0.50	-0.32
t_banagua	0.72	1.00	0.59	0.58	-0.75	-0.69	-0.64	-0.48
t_lixo	0.28	0.59	1.00	0.33	-0.41	-0.39	-0.36	-0.31
t_luz	0.41	0.58	0.33	1.00	-0.36	-0.48	-0.48	-0.31
T_ANALF18M	-0.63	-0.75	-0.41	-0.36	1.00	0.68	0.62	0.45
T_ATRASO_2_BASICO	-0.54	-0.69	-0.39	-0.48	0.68	1.00	0.97	0.61
T_ATRASO_2_FUND	-0.50	-0.64	-0.36	-0.48	0.62	0.97	1.00	0.52
T_ATRASO_2_MED	-0.32	-0.48	-0.31	-0.31	0.45	0.61	0.52	1.00

Figura 8 – Correlação das variáveis
Fonte: Elaborado pelo autor (2022)

Podemos levantar alguns detalhes interessantes, por exemplo: a maior correlação teve valor de 0,97, que ocorreu entre “T_ATRASO_2_FUND” e “T_ATRASO_2_BASICO”. Outro seria a menor correlação (em módulo) como 0,28, observada entre t_lixo e t_agua. Em resumo, todas as correlações ficaram entre moderadas a altas, isso indica que aplicar uma ACP seria eficiente para reduzir as dimensões dos dados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.4 Etapa 2: definir o número de componentes

A segunda etapa tem como objetivo encontrar o melhor número de dimensões que poderão ser resumidas por meio dos dados. No caso, utilizaremos os três critérios: autovalor, *screeplot* e variância explicada.

Antes de selecionar o número de dimensões, precisamos executar o algoritmo, depois disso, verificar os resultados. O primeiro passo é selecionar as variáveis que utilizaremos; no Código-fonte “Seleção das variáveis”, apresentamos o processo. Na Figura “Dados selecionados”, podemos ver os dados filtrados.

```
# separar os dados
cols_pca = ['t_agua', 't_banagua', 't_lixo', 't_luz', 'T_ANALF18M',
            'T_ATRASO_2_BASICO', 'T_ATRASO_2_FUND', 'T_ATRASO_2_MED']

dados_pca = dados[cols_pca]

# dados filtrados
dados_pca.head()
```

Código-fonte 4 – Seleção das variáveis
Fonte: Elaborado pelo autor (2022)

t_agua	t_banagua	t_lixo	t_luz	T_ANALF18M	T_ATRASO_2_BASICO	T_ATRASO_2_FUND	T_ATRASO_2_MED
93.69	80.20	94.05	93.98	13.00	22.03	17.82	7.79
98.54	85.35	96.72	98.58	8.53	19.35	16.74	3.96
95.49	91.63	99.14	96.36	14.73	15.59	12.93	6.93
97.96	93.44	98.17	98.89	8.94	15.57	12.58	7.33
97.53	86.87	91.89	98.85	11.17	12.93	10.27	3.82

Figura 9 – Dados selecionados
Fonte: Elaborado pelo autor (2022)

No segundo passo, teremos que normalizar os dados. Esse processo é essencial na ACP, porque as variáveis podem estar em unidades de medidas diferentes ou a variabilidade nos dados pode ser bastante diferente (o que ocorre em nosso exemplo).

No Código-fonte “Processo para normalizar os dados”, apresentamos um método para normalizar os dados para terem média 0 e desvio-padrão 1, por meio da função “scale”. Na Figura “Dados normalizados”, observamos as variáveis ajustadas,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

percebe-se que temos tanto valores positivos quanto negativos, contudo, isso não muda a estrutura probabilística dos dados.

```
# normalizar
dados_pca_normalizados = scale(dados_pca)
dados_pca_normalizados.shape

# dados ajustados
dados_pca_normalizados
```

Código-fonte 5 – Processo para normalizar os dados
Fonte: Elaborado pelo autor (2022)

```
array([[ 5.49726131e-01, -3.09368550e-02,  2.42172576e-04, ...,
         1.05313573e-01,  7.98869347e-02,  4.30400641e-02],
       [ 8.79209721e-01,  2.06275414e-01,  2.41903242e-01, ...,
        -1.81750931e-01, -5.38453800e-02, -6.33085596e-01],
       [ 6.72008701e-01,  4.95536201e-01,  4.60936871e-01, ...,
        -5.84498146e-01, -5.25623268e-01, -1.08779275e-01],
       ...,
       [-2.36432500e-02,  5.78445343e-01,  4.67272554e-01, ...,
        -3.95978173e-01, -6.74214729e-01, -4.53020799e-01],
       [-5.20246352e-01,  4.25524036e-01,  5.38775268e-01, ...,
         2.53130370e-01,  5.21946531e-01, -6.54269690e-01],
       [ 8.51356469e-01,  6.97281781e-01,  4.36499234e-01, ...,
        -5.09518611e-01, -4.17894459e-01,  1.47946057e-02]])
```

Figura 10 – Dados normalizados
Fonte: Elaborado pelo autor (2022)

Como último passo, vamos executar o algoritmo do ACP. Neste primeiro momento, vamos utilizar as oito colunas e resumir os dados em oito índices. Após esse passo, verificaremos quais dos índices foram relevantes. No Código-fonte “Aplicar o ACP”, apresentamos o processo para executar o algoritmo; neste momento não temos resultados exibidos diretamente na tela do *Colab*.

```
# instanciar o pca
numero_componentes = 8
pca = PCA(n_components=numero_componentes)

# aplicar o algoritmo nos dados
transformacao_pca = pca.fit_transform(dados_pca_normalizados)
```

Código-fonte 6 – Aplicar o ACP
Fonte: Elaborado pelo autor (2022)

Agora que conseguimos treinar o algoritmo, vamos verificar, por meio das três etapas, quantos componentes serão relevantes.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.4.1 Critério do autovalor

O primeiro critério que verificaremos é o do autovalor. A análise é bastante simples, devem-se observar quais componentes apresentaram valores acima de 1. No Código-fonte “Obter os autovalores”, demonstramos a forma de ver os autovalores no Python.

```
# autovalores
autovalores = pca.explained_variance_
autovalores
```

Código-fonte 7 – Obter os autovalores
Fonte: Elaborado pelo autor (2022)

Na Figura “Resultados autovalores”, podemos observar que somente o primeiro componente teve autovalor acima de 1. Diante disso, nesse primeiro critério, reduziríamos os dados a uma única dimensão.

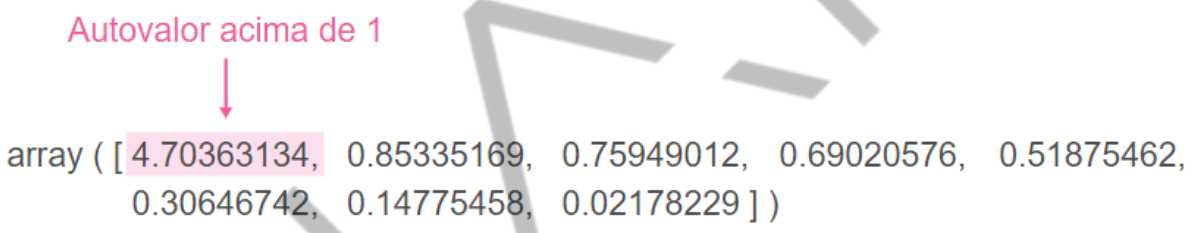

Figura 11 – Resultados autovalores
Fonte: Elaborado pelo autor (2022)

2.4.2 Critério *screeplot*

Como segundo critério, apresentaremos o gráfico de “cotovelo”. Nessa técnica, teremos que verificar o grau de decaimento dos autovalores. No Código-fonte “Obtenção do *screeplot*”, apresentamos o processo para obter o gráfico e selecionar o número de componentes necessários.

```
# componentes
n_componentes = np.arange(pca.n_components_) + 1

# graficos
plt.plot(n_componentes, autovalores, 'o-', linewidth=2, color='blue')
plt.title('Scree Plot')
plt.xlabel('Principal Component')
plt.ylabel('Eigenvalues')
plt.show()
```

Código-fonte 8 – Obtenção do *screeplot*
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na Figura “Análise do *screeplot*”, mostramos os resultados do gráfico. Podemos perceber um decaimento entre o primeiro e o segundo componente. Depois disto, praticamente não há variação entre os demais autovalores. Esses resultados indicam que o primeiro e o segundo componente serão suficientes para resumir os dados.

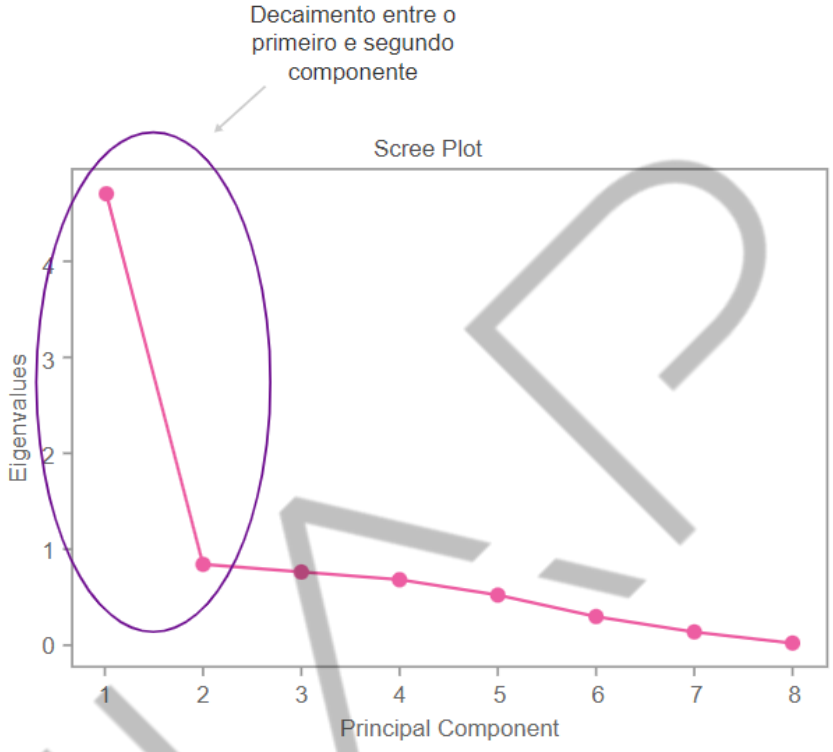

Figura 12 – Análise do *screeplot*
Fonte: Elaborado pelo autor (2022)

2.4.3 Critério da variância explicada

Como terceiro e último critério, avaliaremos a variância explicada. No Código-fonte “Obter variância explicada”, apresentamos como obter a variância explicada individual e a acumulada. Na Figura “Variância explicada demonstrada”, temos os resultados obtidos.

```
# ve individual
pca.explained_variance_ratio_

# ve acumulada
np.cumsum(pca.explained_variance_ratio_)
```

Código-fonte 9 – Obter variância explicada
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Variâncias explicadas individuais

```
array ( [ 0.58784827,  0.10664979,  0.0949192,   0.08626022,  0.06483268,
          0.03830154,  0.018466   ,  0.0027223 ] )
```

Variâncias explicadas acumuladas. Observamos que o primeiro e o segundo componente juntos acumulam quase 70% de explicação.

```
array ( [ 0.58784827,  0.69449806,  0.78941726,  0.87567748,  0.94051016,
          0.9788117 ,  0.9972777 ,  1.          ] )
```

Figura 13 – Variância explicada demonstrada
Fonte: Elaborado pelo autor (2022)

Nesse caso, podemos verificar que, ao utilizar somente o primeiro componente, teremos 58,78% de explicação dos dados, o ideal seria manter 60% no mínimo. Dessa forma, utilizar os dois primeiros componentes será a melhor escolha, uma vez que juntos possuem 69,45% de explicação dos dados.

2.5 Etapa 3: interpretação

Conforme foi citado anteriormente, essa etapa é opcional. Caso o objetivo seja utilizar os componentes em um algoritmo de *Machine Learning* supervisionado, basta adicionar os componentes como novas variáveis. Porém, caso seja interessante criar um índice e apresentar à equipe de negócios, precisaremos entender o que as dimensões significam.

Dado que selecionamos a primeira e a segunda dimensão pelos critérios avaliados anteriormente, teremos que retreinar o algoritmo contendo somente os dois componentes que buscamos. No Código-fonte “Obter a ACP retreinada”, retreinamos a ACP obtendo somente os dois principais componentes.

```
# instanciar o pca
numero_componentes = 2
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
pca = PCA(n_components=numero_componentes)

# retreinar
pca_retreinado = pca.fit_transform(dados_pca_normalizados)
```

Código-fonte 10 – Obter a ACP retreinada
Fonte: Elaborado pelo autor (2022)

Uma vez que temos somente os dois componentes principais de que precisamos, tentaremos interpretar esses índices. Para realizar esse processo, precisamos obter as cargas fatoriais. No Código-fonte “Obtenção das cargas fatoriais”, apresentamos o processo de como obter esses valores.

```
# instanciar o pca
cargas = pca.components_.T * np.sqrt(pca.explained_variance_)
componentes_cols = ['PC'+str(i) for i in range(1, numero_componentes+1
)]
df_cargas = pd.DataFrame(cargas, columns=componentes_cols, index=cols_
pca)

# cargas grafico
df_cargas.style.background_gradient(cmap='coolwarm').set_precision(2)
```

Código-fonte 11 – Obtenção das cargas fatoriais
Fonte: Elaborado pelo autor (2022)

Após execução dos códigos, podemos observar o gráfico apresentado na Figura “Gráfico das cargas modelo retreinado”. Nele temos o quanto a variável está correlacionada com o componente. Percebemos que as variáveis de educação estão correlacionadas com a primeira dimensão, porém não temos valores suficientes para entender a segunda. Diante disso, avaliaremos uma outra técnica.

Sem correlações suficientes para entender o PC2.

	PC1	PC2
t_agua	-0.73	-0.20
t_banagua	-0.90	-0.28
t_lixo	-0.58	-0.52
t_luz	-0.63	-0.22
T_ANALF18M	0.82	0.05
T_ATRASO_2_BASICO	0.90	-0.34
T_ATRASO_2_FUND	0.86	-0.34
T_ATRASO_2_MED	0.65	-0.42

Figura 14 – Gráfico das cargas modelo retreinado
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Diante dessa dificuldade, utilizaremos uma outra função para obter as cargas fatoriais. No Código-fonte “Obtenção dos componentes com *FactorAnalysis*”, retreinaremos o algoritmo utilizando a função *FactorAnalysis*. Depois disso, vamos analisar as cargas fatoriais novamente.

```
# instanciar a fa
numero_componentes = 2
fa = FactorAnalysis(n_components=numero_componentes,
                    rotation = 'varimax',
                    svd_method = 'lapack')

# aplicar
transformacao_fa = fa.fit_transform(dados_pca_normalizados)
```

Código-fonte 12 – Obtenção dos componentes com *FactorAnalysis*
Fonte: Elaborado pelo autor (2022)

Uma vez que retreinamos o algoritmo utilizando a nova função, vamos verificar as novas cargas fatoriais. No Código-fonte “Obtenção das cargas fatoriais com *FactorAnalysis*”, apresentamos o processo para obter as cargas fatoriais.

```
# instanciar a fa
numero_componentes = 2
fa = FactorAnalysis(n_components=numero_componentes,
                    rotation = 'varimax',
                    svd_method = 'lapack')

# aplicar
transformacao_fa = fa.fit_transform(dados_pca_normalizados)
```

Código-fonte 13 – Obtenção das cargas fatoriais com *FactorAnalysis*
Fonte: Elaborado pelo autor (2022)

No Código-fonte “Gerar gráfico das cargas fatoriais com *FactorAnalysis*”, utilizamos um processo para extrair os componentes e gerar um gráfico contendo as correlações dos PC1 e PC2 com as variáveis analisadas.

```
# cargas
cargas = fa.components_.T
componentes_cols = ['PC'+str(i) for i in range(1, numero_componentes+1
)]
df_cargas = pd.DataFrame(cargas, columns=componentes_cols, index=cols_
pca)

# cargas grafico
df_cargas.style.background_gradient(cmap='coolwarm').set_precision(2)
```

Código-fonte 14 – Gerar gráfico das cargas fatoriais com *FactorAnalysis*
Fonte: Elaborado pelo autor (2022)

Na Figura “Resultados cargas fatoriais *FactorAnalysis*”, temos os resultados. Desta vez, pode-se observar claramente a correlação do primeiro componente com as variáveis de educação e o segundo com as variáveis de qualidade de vida. Desse

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

modo, demonstramos que a função foi eficiente para auxiliar na tarefa de interpretar as cargas.

Correlação com variáveis de educação.

Correlação com variáveis de qualidade de vida.

	PC1	PC2
t_agua	-0.35	0.63
t_banagua	-0.42	0.91
t_lixo	-0.22	0.55
t_luz	-0.34	0.48
T_ANALF18M	0.52	-0.59
T_ATRASO_2_BASICO	0.94	-0.33
T_ATRASO_2_FUND	0.93	-0.28
T_ATRASO_2_MED	0.55	-0.27

Figura 15 – Resultados cargas fatoriais *FactorAnalysis*
Fonte: Elaborado pelo autor (2022)

Uma vez que podemos entender os novos componentes obtidos, conseguimos assegurar que houve uma redução de dimensão. No caso, de oito variáveis, podemos explicá-los utilizando dois indicadores: o primeiro relacionado ao atraso na educação e o segundo relacionado à qualidade de vida.

Por fim, podemos adicionar esses novos componentes no banco de dados. No Código-fonte “Adicionar os componentes no *dataframe*”, apresentamos como adicionar esses novos dados e relacioná-los com os estados e municípios. Na Figura “Componentes adicionados aos dados”, apresentamos as novas colunas criadas e que poderão ser utilizadas em outras análises.

```
# adicionar dados
dados[['PC1', 'PC2']] = transformacao_fa
```

Código-fonte 15 – Adicionar os componentes no *dataframe*
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Componentes adicionados

ano	estado	municipio	t_agua	t_banagua	t_lixo	t_luz	T_ANALF18M	T_ATRASO_2_BASICO	T_ATRASO_2_FUND	T_ATRASO_2_MED	PC1	PC2
2010	Rondônia	ALTA FLORESTA D'OESTE	93.69	80.20	94.05	93.98	13.00	22.03	17.82	7.79	0.119588	0.024335
2010	Rondônia	ARIQUEMES	98.54	85.35	96.72	98.58	8.53	19.35	16.74	3.96	-0.130501	0.175610
2010	Rondônia	CABIXI	95.49	91.83	99.14	96.36	14.73	15.59	12.93	6.93	-0.509281	0.311938
2010	Rondônia	CACOAL	97.96	93.44	98.17	98.89	8.94	15.57	12.58	7.33	-0.473412	0.423055
2010	Rondônia	CEREJEIRAS	97.53	86.87	91.89	98.85	11.17	12.93	10.27	3.82	-0.967843	-0.133792

Figura 16 – Componentes adicionados aos dados
Fonte: Elaborado pelo autor (2022)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ATIVIDADE TREINO – ANÁLISE DE COMPONENTES PRINCIPAIS

Esta atividade não é avaliativa. Caso queira ter um feedback de seu tutor, entre em contato com seu tutor pelo Teams

O arquivo “impacto_ambiental.csv” foi disponibilizado e utilizaremos as ferramentas para aplicar uma análise de componentes principais.

O objetivo da análise é avaliar o impacto ambiental do turismo em pequenas comunidades litorâneas. As colunas presentes estão dispostas no quadro a seguir:

Código	Questão
q1	Houve melhoria nos serviços ambientais tais como: gestão de resíduos e remoção de lixo.
q2	Periodicamente é realizada a identificação e correção dos problemas ambientais na comunidade.
q3	Turismo colabora para a preservação da biodiversidade.
q4	Comunidade tem consciência da importância da conservação do meio ambiente.
q5	A eliminação de resíduos é feita de maneira ecológica e não poluente.
q5	Comunidade colabora para limpeza e conservação de trilhas e eliminação da poluição visual.

Para esta atividade, analise os dados propostos e responda às perguntas a seguir. Ao final, deve ser entregue um arquivo “.ipynb”, contendo os códigos e as respostas.

1. Acesse o *Colab*, carregue os módulos e os dados necessários.

2. Realize uma preparação de dados:

   a) Filtre os nulos.

   b) Normalize os dados.

3. Avalie as correlações: elas são altas ou baixas?

4. Defina o número de componentes pelos critérios:

   a) Autovalor

   b) Screeplot

   c) Variância explicada

5. Quantos componentes podem ser adotados?

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Importante: as respostas estão disponibilizadas no arquivo anexo “.ipynb”.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

HAIR, J. F. *et al.* **Análise multivariada de dados**. São Paulo: Bookman, 2009.

JOLLIFFE, I. T.; CADIMA, J. Principal component analysis: a review and recent developments. **Philosophical Transactions of the Royal Society A,** v. 374, n. 2.065, p. 1-16, 2016. Disponível em: http://dx.doi.org/10.1098/rsta.2015.0202. Acesso em: 23 jan. 2022.

MAROCO, J. **Análise estatística com utilização do SPSS.** Lisboa: Sílabo, 2010.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com